

Checkout v7

What's new

- Safer fork pull request handling: checkout now refuses to check out fork pull request code by default when the workflow is triggered by `pull_request_target` or `workflow_run`. These triggers run with the base repository's `GITHUB_TOKEN`, secrets, and runner access, where executing a fork's code commonly leads to "pwn request" vulnerabilities.
  - To opt in after reviewing the risks, set the new `allow-unsafe-pr-checkout: true` input.
- Migrated `actions/checkout` to ESM to support new versions of the `@actions/*` packages.
- Updated direct and transitive dependencies, including security fixes for known vulnerabilities.

Checkout v6

What's new

- Improved credential security: `persist-credentials` now stores credentials in a separate file under `$RUNNER_TEMP` instead of directly in `.git/config`
- No workflow changes required — `git fetch`, `git push`, etc. continue to work automatically
- Running authenticated git commands from a Docker container action requires Actions Runner v2.329.0 or later

Checkout v5

What's new

- Updated to the node24 runtime
  - This requires a minimum Actions Runner version of v2.327.1 to run.

Checkout v4

This action checks-out your repository under `$GITHUB_WORKSPACE`, so your workflow can access it.

Only a single commit is fetched by default, for the ref/SHA that triggered the workflow. Set `fetch-depth: 0` to fetch all history for all branches and tags. Refer here to learn which commit `$GITHUB_SHA` points to for different events.

The auth token is persisted in the local git config. This enables your scripts to run authenticated git commands. The token is removed during post-job cleanup. Set `persist-credentials: false` to opt-out.

When Git 2.18 or higher is not in your PATH, falls back to the REST API to download the files.

Note

Thank you for your interest in this GitHub action, however, right now we are not taking contributions.

We continue to focus our resources on strategic areas that help our customers be successful while making developers' lives easier. While GitHub Actions remains a key part of this vision, we are allocating resources towards other areas of Actions and are not taking contributions to this repository at this time. The GitHub public roadmap is the best place to follow along for any updates on features we're working on and what stage they're in.

We are taking the following steps to better direct requests related to GitHub Actions, including:

1. We will be directing questions and support requests to our Community Discussions area
2. High Priority bugs can be reported through Community Discussions or you can report these to our support team https://support.github.com/contact/bug-report.
3. Security Issues should be handled as per our security.md

We will still provide security updates for this project and fix major breaking changes during this time.

You are welcome to still raise bugs in this repo.

What's new

Please refer to the release page for the latest release notes.

Usage

```
- uses: actions/checkout@v7
  with:
    # Repository name with owner. For example, actions/checkout
    # Default: ${{ github.repository }}
    repository: ''

    # The branch, tag or SHA to checkout. When checking out the repository that
    # triggered a workflow, this defaults to the reference or SHA for that event.
    # Otherwise, uses the default branch.
    ref: ''

    # Personal access token (PAT) used to fetch the repository. The PAT is
configured
    # with the local git config, which enables your scripts to run authenticated
git
    # commands. The post-job step removes the PAT.
    #
    # We recommend using a service account with the least permissions necessary.
```

```
Also
    # when generating a new PAT, select the least scopes necessary.
    #
    # [Learn more about creating and using encrypted
secrets](https://help.github.com/en/actions/automating-your-workflow-with-github-
actions/creating-and-using-encrypted-secrets)
    #
    # Default: ${{ github.token }}
    token: ''

    # SSH key used to fetch the repository. The SSH key is configured with the
local
    # git config, which enables your scripts to run authenticated git commands.
The
    # post-job step removes the SSH key.
    #
    # We recommend using a service account with the least permissions necessary.
    #
    # [Learn more about creating and using encrypted
secrets](https://help.github.com/en/actions/automating-your-workflow-with-github-
actions/creating-and-using-encrypted-secrets)
    ssh-key: ''

    # Known hosts in addition to the user and global host key database. The public
SSH
    # keys for a host may be obtained using the utility `ssh-keyscan`. For
example,
    # `ssh-keyscan github.com`. The public key for github.com is always implicitly
    # added.
    ssh-known-hosts: ''

    # Whether to perform strict host key checking. When true, adds the options
    # `StrictHostKeyChecking=yes` and `CheckHostIP=no` to the SSH command line.
Use
    # the input `ssh-known-hosts` to configure additional hosts.
    # Default: true
    ssh-strict: ''

    # The user to use when connecting to the remote SSH host. By default 'git' is
    # used.
    # Default: git
    ssh-user: ''

    # Whether to configure the token or SSH key with the local git config
    # Default: true
    persist-credentials: ''

    # Relative path under $GITHUB_WORKSPACE to place the repository
    path: ''

    # Whether to execute `git clean -ffdx && git reset --hard HEAD` before
fetching
    # Default: true
```

```
    clean: ''

    # Partially clone against a given filter. Overrides sparse-checkout if set.
    # Default: null
    filter: ''

    # Do a sparse checkout on given patterns. Each pattern should be separated
with
    # new lines.
    # Default: null
    sparse-checkout: ''

    # Specifies whether to use cone-mode when doing a sparse checkout.
    # Default: true
    sparse-checkout-cone-mode: ''

    # Number of commits to fetch. 0 indicates all history for all branches and
tags.
    # Default: 1
    fetch-depth: ''

    # Whether to fetch tags, even if fetch-depth > 0.
    # Default: false
    fetch-tags: ''

    # Whether to show progress status output when fetching.
    # Default: true
    show-progress: ''

    # Whether to download Git-LFS files
    # Default: false
    lfs: ''

    # Whether to checkout submodules: `true` to checkout submodules or `recursive`
to
    # recursively checkout submodules.
    #
    # When the `ssh-key` input is not provided, SSH URLs beginning with
    # `git@github.com:` are converted to HTTPS.
    #
    # Default: false
    submodules: ''

    # Add repository path as safe.directory for Git global config by running `git
    # config --global --add safe.directory <path>`
    # Default: true
    set-safe-directory: ''

    # The base URL for the GitHub instance that you are trying to clone from, will
use
    # environment defaults to fetch from the same instance that the workflow is
    # running from unless specified. Example URLs are https://github.com or
    # https://my-ghes-server.example.com
```

```
    github-server-url: ''

    # Required to check out fork pull request code from a workflow triggered by
    # `pull_request_target` or `workflow_run`. These workflows run with the base
    # repository's GITHUB_TOKEN, secrets, default-branch cache scope, and runner
    # access; fetching and executing a fork's code in that trusted context
commonly
    # leads to "pwn request" vulnerabilities. Set to `true` only after reviewing
the
    # risks at https://gh.io/securely-using-pull_request_target.
    # Default: false
    allow-unsafe-pr-checkout: ''
```

Scenarios

- Checkout V5
  - What's new
- Checkout V4
  - Note
- What's new
- Usage
- Scenarios
  - Fetch only the root files
  - Fetch only the root files and `.github` and `src` folder
  - Fetch only a single file
  - Fetch all history for all tags and branches
  - Checkout a different branch
  - Checkout HEAD^
  - Checkout multiple repos (side by side)
  - Checkout multiple repos (nested)
  - Checkout multiple repos (private)
  - Checkout pull request HEAD commit instead of merge commit
  - Checkout pull request on closed event
  - Push a commit using the built-in token
  - Push a commit to a PR using the built-in token
- Recommended permissions
- License

Fetch only the root files

```
- uses: actions/checkout@v7
  with:
    sparse-checkout: .
```

Fetch only the root files and `.github` and `src` folder

```
- uses: actions/checkout@v7
  with:
    sparse-checkout: |
      .github
      src
```

Fetch only a single file

```
- uses: actions/checkout@v7
  with:
    sparse-checkout: |
      README.md
    sparse-checkout-cone-mode: false
```

Fetch all history for all tags and branches

```
- uses: actions/checkout@v7
  with:
    fetch-depth: 0
```

Checkout a different branch

```
- uses: actions/checkout@v7
  with:
    ref: my-branch
```

Checkout HEAD^

```
- uses: actions/checkout@v7
  with:
    fetch-depth: 2
- run: git checkout HEAD^
```

Checkout multiple repos (side by side)

```
- name: Checkout
  uses: actions/checkout@v7
  with:
    path: main

- name: Checkout tools repo
  uses: actions/checkout@v7
  with:
    repository: my-org/my-tools
    path: my-tools
```

> - If your secondary repository is private or internal you will need to add the option noted in Checkout multiple repos (private)

Checkout multiple repos (nested)

```
- name: Checkout
  uses: actions/checkout@v7

- name: Checkout tools repo
  uses: actions/checkout@v7
  with:
    repository: my-org/my-tools
    path: my-tools
```

> - If your secondary repository is private or internal you will need to add the option noted in Checkout multiple repos (private)

Checkout multiple repos (private)

```
- name: Checkout
  uses: actions/checkout@v7
  with:
    path: main

- name: Checkout private tools
  uses: actions/checkout@v7
  with:
    repository: my-org/my-private-tools
    token: ${{ secrets.GH_PAT }} # `GH_PAT` is a secret that contains your PAT
    path: my-tools
```

> - `${{ github.token }}` is scoped to the current repository, so if you want to checkout a different repository that is private you will need to provide your own PAT.

Checkout pull request HEAD commit instead of merge commit

```
- uses: actions/checkout@v7
  with:
    ref: ${{ github.event.pull_request.head.sha }}
```

Checkout pull request on closed event

```
on:
  pull_request:
    branches: [main]
    types: [opened, synchronize, closed]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
```

Push a commit using the built-in token

```yaml
on: push
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - run: |
          date > generated.txt
          # Note: the following account information will not work on GHES
          git config user.name "github-actions[bot]"
          git config user.email "41898282+github-
actions[bot]@users.noreply.github.com"
          git add .
          git commit -m "generated"
          git push
```

NOTE: The user email is `{user.id}+{user.login}@users.noreply.github.com`. See users API: https://api.github.com/users/github-actions%5Bbot%5D

Push a commit to a PR using the built-in token

In a pull request trigger, `ref` is required as GitHub Actions checks out in detached HEAD mode, meaning it doesn't check out your branch by default.

```yaml
on: pull_request
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
        with:
          ref: ${{ github.head_ref }}
      - run: |
          date > generated.txt
          # Note: the following account information will not work on GHES
          git config user.name "github-actions[bot]"
          git config user.email "41898282+github-
actions[bot]@users.noreply.github.com"
          git add .
          git commit -m "generated"
          git push
```

NOTE: The user email is `{user.id}+{user.login}@users.noreply.github.com`. See users API: https://api.github.com/users/github-actions%5Bbot%5D

Recommended permissions

When using the `checkout` action in your GitHub Actions workflow, it is recommended to set the following `GITHUB_TOKEN` permissions to ensure proper functionality, unless alternative auth is provided via the `token` or `ssh-key` inputs:

```
permissions:
  contents: read
```

License

The scripts and documentation in this project are released under the MIT License